

Internet Programming I

Chapter 4 JavaScript: Array in Java Script



Group: 3
3rd Year Sec B Students,
Dept. of Software Eng.

Jan 2022, AASTU

Objectives



After success completion of the session you will be able to:

- Understand declaration and initialization of array in JavaScript
- Understand the how the array methods work
- Manipulate arrays by using JavaScript built in methods

Array

- Declaration and Initialization of arrays
- Array Element Manipulation
- Array Methods in JavaScript

1. What are arrays in JS?

- ✓ An array is a collection of similar data elements stored at contiguous memory locations
- ✓ But in JavaScript, an array is an ordered list of values either similar or mixed in data type.
- ✓ Arrays are list-like objects whose prototype has methods to perform traversal and mutation operations.
- ✓ In java script arrays are considered to be type of objects.
- ✓ The difference between arrays and objects is:
 - **Arrays**: use numbered indexes.
 - **Objects**: use named indexes.

2. Declaration and initialization

We have two methods to declare an array in JavaScript

- ❑ Using array literals

`let/const/var` arrayName = [list1,list2.....listN];

- ❑ By using array constructor

`let/const/var` arrayName = `new` Array(list1,list2..listN);

NOTE: We can use both declaration methods but for simplicity we use the first one mostly.

3. Manipulation of array elements

✓ Accessing and assigning value

In similar way with other programming languages we use bracket notation to assign and access array elements in JavaScript.

Example

```
const cars = [ "Saab", "Volvo", "BMW" ]; //assigning  
let car = cars[0]; //accessing one element  
cars[1] = "Toyota"; // assigning value
```

We can also use built in functions to do the same thing

push() : for appending element at the end

pop() : for removing and returning value at the top

Array element manipulation example

```
const numbers = []; // array declaration with literals
const peoples = new Array("John","jack","Brian"); // array declaration with
array constructor
for(let i=0;i<4;i++){
    numbers[i] = (i+2);
}
document.write(numbers + "<br>"); // will write the whole array into the
document as
//list of numbers separated by commalike 2,3,4,5
document.write(numbers[0] + "<br>"); // accessing array element
numbers[0] = 10; // assigning value to an element using index notation
document.write(numbers[0] + "<br>"); //prints 10
numbers.push(25); //add 25 at the end of the array
document.write(numbers + "<br>"); //prints 10,3,4,5,25
document.write(numbers.pop()); // removes the last element return it
```

Example cont'd

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Arrays</h2>

<p>The push method appends a new element to an array.</p>

<button onclick="myFunction()">Try it</button>

<p id="demo"></p>

<script>
const fruits = ["Banana", "Orange", "Apple"];
document.getElementById("demo").innerHTML = fruits;

function myFunction() {
  fruits.push("Lemon");
  document.getElementById("demo").innerHTML = fruits;
}
</script>

</body>
</html>
```

FIRST OUTPUT

JavaScript Arrays

The push method appends a new element to an array.

Try it

Banana,Orange,Apple

AFTER PRESSING TRY IT BUTTON

JavaScript Arrays

The push method appends a new element to an array.

Try it

Banana,Orange,Apple,Lemon

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Arrays</h2>

<p id="demo"></p>

<script>
const cars = ["Saab", "Volvo", "BMW"];
document.getElementById("demo").innerHTML = cars;
</script>

</body>
</html>
```

OUTPUT

JavaScript Arrays

Saab,Volvo,BMW

Fig: Accessing entire array element

Traversing over arrays

We can use looping statements to iterate over arrays.

1. Indexed for loop

Example :

```
const peoples = new  
    Array("John", "Alex", "Brian", "Abebe", "Bekele", "Nati", "Nardos");  
for(let i=0; i<10; i++){  
    console.log(peoples[i]);  
}
```

2. for in loop : are used to loop through the properties of an Object, meaning gives us the keys to iterate over the object.

Syntax

```
for(let itrVar in ArrayName){  
    // any operation  
} itrVar will contain the key for the array
```

Traversing over arrays cont'd

Let's use the above example

```
for(let key in peoples){  
    console.log(peoples[key]); // prints elements of  
    peoples to the console  
}
```

3. For of loop : used to traverse over the values of the object. Unlike the for in loop which iterates over the keys of the object, for of loop accesses the values of the objects.

Syntax

```
for(let itrVar of ArrayName){  
    // any operation  
} itrVar will contain the values in the array
```

4. Array Methods

JavaScript has many built in methods to manipulate arrays.

Methods for array element addition and removal operations

- `push()` : append element at the end
- `pop()`: removes and returns value at the end
- `splice()`: to add and/or remove elements
- `shift()` : removes the first element
- `unshift()` : add element at the beginning

Method	syntax	parameter	Return value
<code>push()</code>	<code>array.push(value)</code>	Value to be inserted	New length of the array
<code>pop()</code>	<code>array.pop()</code>	No parameter	Value of the popped element
<code>shift()</code>	<code>array.shift()</code>	No parameter	Remove and return the first element
<code>unshift()</code>	<code>array.unshift(v1,v2,..vn)</code>	Infinite number of values to be inserted,at least one	New array length
<code>splice()</code>	<code>array.splice(start index, how many, item1,item2...itemN)</code>	- Index from which removal starts => required -How many => number of items to be removed(optional) -items to be added in place of the deleted items => optional	Array containing removed items

5. Array Methods cont'd

Methods to create array from an array

- `copyWithin()`
- `slice()`
- `Array.from()`
- `concat()`

Example

```
const peoples =  
new Array("John",  
"Alex",  
"Brian",  
"Abebe",  
"Bekele",  
"Nati", "Nardos");
```

Method	syntax	parameter	Return value
<code>copyWithin()</code>	<code>array.copyWithin(from index, to index)</code>	Start index and ending index	Array containing the elements with in the specified index interval
<code>slice()</code>	<code>array.slice(from index, to index)</code>	Start index and ending index(not included)	Array containing the elements with in the specified index interval
<code>Array.from()</code>	<code>Array.from(string)</code>	String from which we want to create an array	Array of the string inerted
<code>concat()</code>	<code>rootArray.concat(arr1,arr2,..arrN)</code>	Arrays to be concatenated with the root array	New array

```
peoples.push("Gigi"); //add Gigi at the end
peoples.pop(); // removes and return Gigi
peoples.shift(); // removes and return first element => John
peoples.unshift("Bethel"); // insert Bethel at the beginning
document.write("<br>" + peoples + "<br>"); // prints
Bethel,Alex,Brian,Abebe,Bekele,Nati,Nardos
document.write(peoples.slice(0,2) + "<br>"); // prints Bethel,Alex
peoples.concat("Yonnas","Honok"); // concatenates all the arrays into one array
and return it
```

6. cont'd

Sorting functions

- `sort()`
- `reverse()`

Method	syntax	parameter	Return value
<code>sort()</code>	<code>array.sort()</code>	Optional compare function	Returns the array in sorted form
<code>reverse()</code>	<code>array.reverse()</code>	No parameter	Array in reversed order to the original array

7. cont'd

Methods used to implement a function on each element

- every() :
- filter()
- foEach()
- map()

Method	syntax	parameters	Return type
every()	array.every(functionName)	Function for test	True if all items pass the test,else false
filter()	array.filter(functionName)	Function for test	Array containing the elements which passed the test
forEach()	array.forEach(functionName)	Function to act on each elements	no
map()	array.map(functionName)	Function that performs the manipulation on items	Return new array
some()	array.some(function)	Function for test	true,false(boolean) True if at least one element satisfies the test condition
reduce()	array.reduce(function)	Function for test	One reduced value for elements

Example

```
var total = 0;

function MoreThan6(item){
    return item > 6;
}

function double(value){
    return value*2;
}

function sum(value){
    total += value;
}

function startWith(item){
    return item.toLowerCase().startsWith("a");
}
```

Cont'd

```
const result = numbers.filter(MoreThan6);  
document.write(result + "<br>"); // will print 10,7,8,9,10,11  
document.write(numbers.map(double) + "<br>"); //  
    20,6,8,10,12,14,16,18,20,22  
document.write(numbers.every(MoreThan6) + "<br>"); // return false  
numbers.forEach(sum);  
document.write(total + "<br>"); // print 73  
document.write(peoples.filter(startWith)); // Alex,Abebe
```

Array methods cont'd

Methods for searching

- `find()`
- `indexOf()`
- `lastIndexOf()`
- `findIndex()`

Method	syntax	parameters	Return type
<code>find()</code> Finds an element that satisfies the test condition	<code>array.find(function)</code>	Function containing test condition	Returns first element that passes the test in the function, else return undefined
<code>includes()</code> Checks the existence of an element	<code>array.includes(value)</code>	Value to be searched	Boolean True, if exists false, if not exist
<code>indexOf()</code>	<code>array.indexOf(value)</code>	Value to be searched	if found return Index of value else -1
<code>lastIndexOf()</code>	<code>array.lastIndexOf(value)</code>	Value to be searched	Starts search from the last element if found return Index of value else -1
<code>findIndex()</code>	<code>array.findIndex(function)</code>	Function for test condition	Return the index of the element for the value that passes the test,else return -1

Example

```
const word = peoples.find((value)=>{return  
  value.startsWith("B")}); // returns first match element starts  
  with 'B'  
console.log(word); // print ot console of the broser  
const index = peoples.findIndex((value)=>{return  
  value.endsWith("x")}); // returs index of the element that  
  matches the condition,1  
console.log(peoples[index]); // prints value at the index returned  
  by the above method => Alex  
const index2 = peoples.indexOf("Nati"); // starts search from the  
  begining and returns index of 'Nati' if exist else -1 (returns 2)  
const index3 = peoples.lastIndexOf("Nati"); // starts search from  
  the end and returns index of 'Nati' if exist else -1 (returns 6)  
const isFound = peoples.includes("Peter"); // returns true if  
  exist else false ==> (retrurns false)
```

Array prototype

In javascript the Array object has a property called **prototype** that is used to allow to add new methods and properties to the **Array()** object. syntax

```
Array().prototype.Name = value;
```

new property or method name

function definition that performs the new functionality

For example if we want to expand the functionality of the object by adding a method that converts all the elements of an array of strings to uppercase or lowercase we can add our custom method in to the object.

Example

```
const peoples = new
Array("John", "Alex", "Nati", "Brian", "Abebe", "Bekele", "Nati", "Nardos"); // array
declaration with array constructor
Array.prototype.toUpperCase = function() {
    for (let i = 0; i < this.length; i++) {
        this[i] = this[i].toUpperCase(); // `this` refers the object later
will be substituted by the array
    }
}; // custom method to convert all the elements of an array of string to
upper case
peoples.toUpperCase(); // upperCase() method called
console.log(peoples); // prints this to console ['JOHN', 'ALEX', 'NATI', 'BRIAN',
'ABEBE', 'BEKELE', 'NATI', 'NARDOS']
```

Demosnatration Example code

```
<div class="main">  
  <div class="input-con">  
    <label for="sub1">subject 1</label>  
    <input type="text" name="mark1" id="sub1">  
  </div>
```

```
  <div class="input-con">  
    <label for="sub2">subject 2</label>  
    <input type="text" name="mark2" id="sub2">  
  </div>
```

```
  <div class="input-con">  
    <label for="sub3">subject 3</label>  
    <input type="text" name="mark3" id="sub3">  
  </div>
```

```
  <div class="input-con">  
    <label for="sub4">subject 4</label>  
    <input type="text" name="mark4" id="sub4">  
  </div>
```

```
  <div class="input-con">  
    <label for="sub5">subject 5</label>  
    <input type="text" name="mark5" id="sub5">  
  </div>
```

```
  <div class="input-con">  
    <label for="sub6">subject 6</label>  
    <input type="text" name="mark6" id="sub6">  
  </div>
```

```
<div class="btn">  
  <button type="button" id = "btn-1">Save</button>  
</div>
```

```
<div class="btn">  
  <button type="button" id = "btn-2" title = "reduce() to sum value">Total</button>  
</div>
```

```
<div class="btn">  
  <button type="button" id = "btn-3">Average</button>  
</div>
```

```
<div class="btn">  
  <button type="button" id = "btn-4" title="subarray of marks > 90 map()/filter()">Marks more than 90</button>  
</div>
```

```
<div class="btn">  
  <button type="button" id = "btn-5" title="find if 100? find()/indexOf()">is there 100?</button>  
</div>
```

```
<div class="btn">  
  <button type="button" id = "btn-6" title="sorting increasing order sort()">increasing order</button>  
</div>
```

Demosnatration Example code

```
<div class="btn">
  <button type="button" id = "btn-7" title="reverseSort() custom method">Decreasing
order</button>
</div>
<textarea name="result" id="result" cols="30" rows="10"></textarea>
</div>
<script>
Array.prototype.reverseSort = (marks)=>{marks.sort(function(a,b){return b-a;}); return marks;} // customized
method to sort in descending order
var inputs = document.querySelectorAll("input");
  console.log(inputs);
  const save = document.getElementById("btn-1");
  const total = document.getElementById("btn-2");
  const average = document.getElementById("btn-3");
  const More90 = document.getElementById("btn-4");
  const find100 = document.getElementById("btn-5");
  const sort = document.getElementById("btn-6");
  const reverse = document.getElementById("btn-7");
  const write = document.getElementById("result");
```

Demosnatration Example code

```
var marks = [];  
console.log(marks);  
function store(){  
    marks = [];  
    for(let mark of inputs){  
        marks.push(Number(mark.value));  
    }  
    console.log(marks);  
}  
  
function findValue(value){  
    return value == 100;  
}  
  
function moreThan90(value){  
    return value > 90;  
}  
  
function sum(sum,value){  
    return (sum) + (value);  
}  
save.setAttribute("onclick","store()");
```

Demosnatration Example code

```
total.onclick = ()=>{
    write.innerText = "Total mark = " + marks.reduce(sum,0);
}

average.onclick = ()=>{
    write.innerText = "Average mark = " + (marks.reduce(sum,0))/6;
}

More90.onclick = ()=>{
    write.innerText = "marks more than 90 = " + marks.filter(moreThan90);
}

find100.onclick = ()=>{
    write.innerText = "is there any mark of 100?" + `at index ${marks.findIndex(findValue)}`; // marks
}

sort.addEventListener("click", ()=>{
    marks.sort((a,b)=>{return a-b});
    write.innerText = "sorted in increasing order: " + marks;
});

reverse.addEventListener("click", ()=>{
    marks.reverseSort(marks);
    write.innerText = "sorted in decreasing order: " + marks;
})

</script>
```

Demosnatration Example Output

subject 1	90	subject 2	67	subject 3	100
subject 4	96	subject 5	99	subject 6	78

Total mark = 530

Demosnatration Example Output

subject 1 subject 2 subject 3
subject 4 subject 5 subject 6

Save

Total

Average

Marks more than 90

is there 100?

increasing order

Decreasing order

Average mark = 88.33333333333333

Demosnatration Example Output

subject 1 subject 2 subject 3
subject 4 subject 5 subject 6

Save

Total

Average

Marks more than 90

is there 100?

increasing order

Decreasing order

marks more than 90 = 100,96,99

Demosnatration Example Output

subject 1	90	subject 2	67	subject 3	100
subject 4	96	subject 5	99	subject 6	78

```
is there any mark of 100?at  
index 2
```

Demosnatration Example Output

subject 1 subject 2 subject 3
subject 4 subject 5 subject 6

Save

Total

Average

Marks more than 90

is there 100?

increasing order

Decreasing order

```
sorted in increasing order:  
67,78,90,96,99,100
```

Demosnatration Example Output

subject 1 subject 2 subject 3
subject 4 subject 5 subject 6

Save

Total

Average

Marks more than 90

is there 100?

increasing order

Decreasing order

```
sorted in decreasing order:  
100,99,96,90,78,67
```

Reading Resources/Materials

Chapter X (if you used text book)

- ✓ Jon Duckett; HTML and CSS Design and Build Websites, 2011 John Wiley & Sons, Inc., Indianapolis, Indiana

Links

- ✓ Paul Deitel,

Thank You
For Your Attention!!

Any Questions

